High Level Design

On

IAC Provisioning for Telecomm System

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Mohd Dilshad Khan	EN22CS301602
02	Nishit Sharma	EN22CS301658
03	Paridhi Gupta	EN22CS301683
04	Nirmal Kant Shrivastava	EN22CS301656
05	Mohd Suhail Khan	EN22CS301604

Group Name : Group 08D6

Project Number: DO-19

Industry Mentor Name: Vaibhav

University Mentor Name: Prof. Akshay Saxena

Academic Year: 2025-2026

1. Introduction

1.1. Scope of the document

This document defines the high-level architecture and overall design of the **IaC-based provisioning system for the Telecom infrastructure**. It outlines the structure, components, and interactions required to automate the deployment and management of telecom services using modern DevOps practices.
The scope includes the use of **Infrastructure as Code (IaC)** tools such as Terraform for provisioning resources, Docker for containerization, and Kubernetes for orchestration. It also covers the integration of CI/CD pipelines using Jenkins to enable automated build, testing, and deployment processes.
Additionally, this document focuses on ensuring **scalability, consistency, and reliability** in infrastructure deployment by eliminating manual configurations and enabling repeatable environments across development and production.

1.2. Intended Audience

This document is intended for stakeholders involved in the design, development, deployment, and evaluation of the IaC-based Telecom provisioning system.

- **Project Supervisors:** To assess the overall system architecture, design feasibility, and alignment with project objectives.
- **DevOps Engineers:** To understand the automation workflow, CI/CD pipeline integration, and infrastructure provisioning using tools like Terraform, Docker, and Kubernetes.
- **System Architects:** To review the architectural design, component interactions, and scalability considerations of the system.
- **Developers:** To gain a high-level understanding of how infrastructure and application components are integrated and deployed.

1.3. System overview

The system is designed to automate the provisioning and management of telecom infrastructure using an **Infrastructure as Code (IaC)** approach. It enables developers and DevOps engineers to define, deploy, and manage infrastructure resources in a consistent and repeatable manner.

The architecture integrates tools such as **Terraform** for infrastructure provisioning, **Docker** for containerization, and **Kubernetes** for orchestration of telecom microservices. A **CI/CD pipeline using Jenkins** is used to automate the

process of building, testing, and deploying the infrastructure and application components.

The workflow begins with developers writing IaC scripts and pushing them to a Git repository. This triggers the CI/CD pipeline, which executes Terraform scripts to provision resources on local or cloud environments. The deployed infrastructure hosts telecom services such as billing and network management in containerized form.

2. System Design

2.1. Application Design

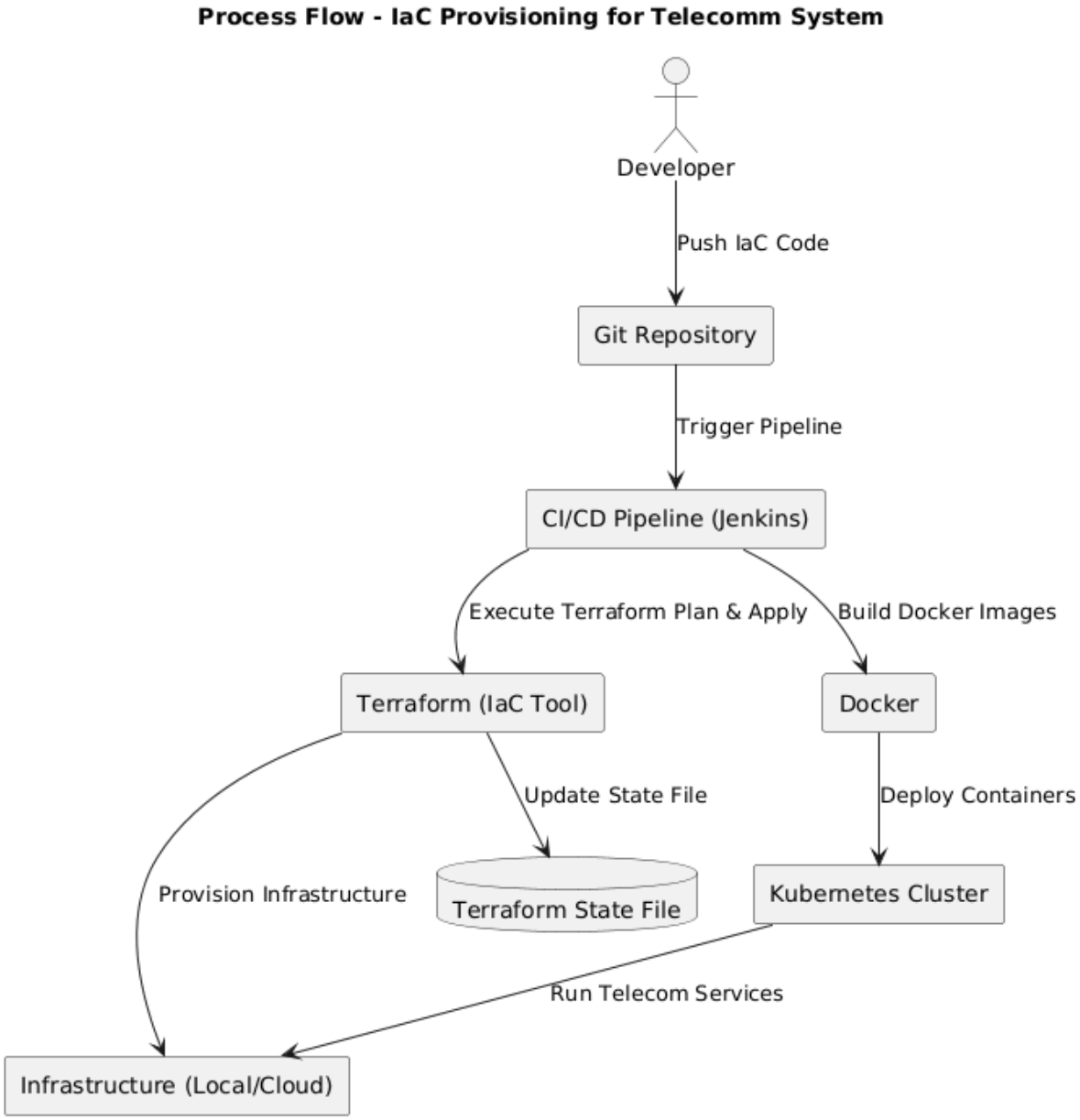
The application is designed using a **microservices architecture**, where telecom functionalities are divided into independent and loosely coupled services. Each service is responsible for a specific domain such as billing, network management, or user operations, allowing easier development, deployment, and scaling.

These services are containerized using **Docker**, ensuring that each microservice runs in an isolated and consistent environment. The containers are then orchestrated using **Kubernetes**, which manages deployment, scaling, load balancing, and fault tolerance across the system.

2.2. Process Flow

The process flow describes the sequence of steps involved in automating the provisioning and deployment of telecom infrastructure using Infrastructure as Code (IaC) and DevOps practices.

The process begins at the **developer's machine**, where IaC scripts and configuration files are created using tools such as Terraform. These scripts define the infrastructure requirements, including compute resources, networking, and container environments.



2.3. Information Flow

The information flow describes how data moves across system components in the IaC-based telecom provisioning system. Infrastructure code is pushed to the Git repository, which triggers the CI/CD pipeline (Jenkins). Jenkins processes the code and invokes Terraform to provision or update infrastructure resources. Terraform maintains a state file to track system changes. Simultaneously, Jenkins builds Docker images and deploys them to the Kubernetes cluster. This ensures smooth communication between components, consistent infrastructure management, and accurate tracking of system state.

2.4. Components Design

- **Git Repository**: Serves as the single source of truth for infrastructure code.
- **Jenkins/CI-CD**: Acts as the automation controller that triggers deployments upon code changes.
- **Terraform**: The core IaC tool used to provision and manage resources.
- **Docker & Kubernetes**: The runtime environment for hosting telecom microservices.

2.5. Key Design Considerations

- **Modularity:** IaC code is divided into reusable modules (e.g., network, compute, services) for better maintainability.
- **Scalability:** Kubernetes enables automatic scaling of telecom services based on demand.
- **Automation:** CI/CD pipelines reduce manual intervention and ensure faster deployments.
- **Consistency:** Docker ensures uniform environments across development and production.

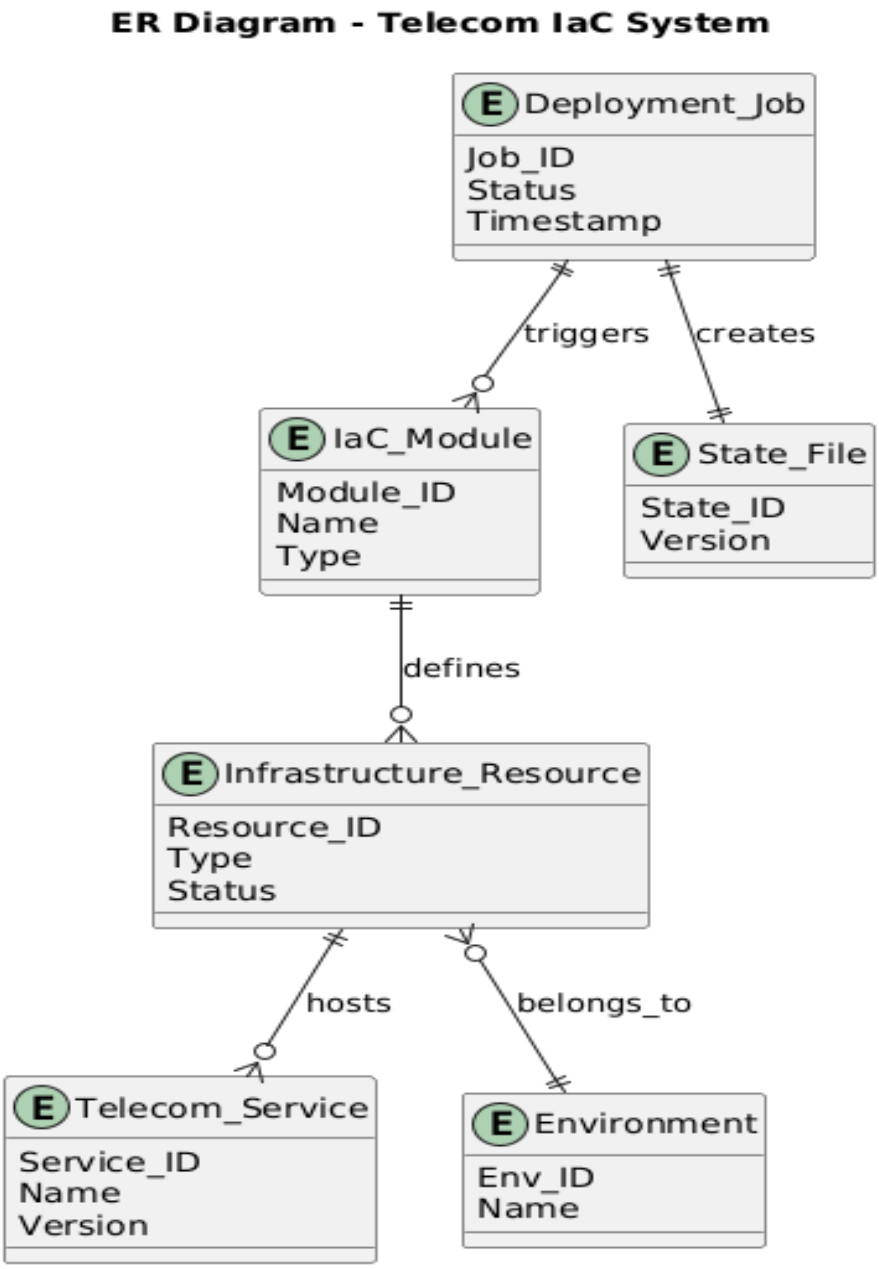
2.6. API Catalogue

- **Kubernetes API**: Used by the CI/CD pipeline to communicate with the cluster and manage container lifecycles.

3. Data Design

3.1. Data Model

The system follows an **Infrastructure Meta-Model** to represent and manage the relationships between different entities involved in the provisioning process. The key entities include **IaC_Modules**, **Deployment_Jobs**, **Infrastructure_Resources**, and **Telecom_Services**.



3.2. Data Access Mechanism

Access to infrastructure data and configuration is managed through secure and controlled mechanisms. The Git repository provides version-controlled access to IaC scripts, ensuring only authorized users can modify infrastructure code. Terraform uses backend storage (such as remote state files) to securely store and retrieve infrastructure state. Access to these state files is controlled using authentication mechanisms like IAM roles or access keys. Additionally, CI/CD tools like Jenkins enforce access control policies, ensuring that only authorized users can trigger deployment processes.

3.3. Data Retention Policies

The system maintains data to ensure reliability, traceability, and disaster recovery.

- **State History:** Previous versions of the Terraform state file are retained to allow rollback in case of failures.
- **Audit Logs:** CI/CD pipeline logs (Jenkins) are stored to track deployment activities and user actions.
- **Version Control:** Git maintains history of all infrastructure code changes for future reference.

3.4. Data Migration

Infrastructure migration is handled using Terraform by updating configuration settings. By modifying the **provider configuration**, resources can be migrated between environments such as local systems and cloud platforms (e.g., AWS). Terraform ensures that the infrastructure state remains consistent during migration, minimizing downtime and preventing configuration drift.

4. Interfaces

- **User Interface (UI):** A centralized web dashboard provides visibility into deployments, infrastructure status, logs, and service health. It allows users to trigger deployments, monitor pipelines, and manage telecom services.
- **API Interface:** REST APIs are used for communication between services and external systems. These APIs enable automation, integration with third-party tools, and programmatic control over infrastructure and services.
- **Command Line Interface (CLI):** Developers and DevOps engineers use CLI tools like Terraform, Docker, and Git for scripting, debugging, and managing infrastructure resources.
- **System-to-System Interface:** Internal communication between components (Jenkins, Terraform, Kubernetes) occurs through APIs and service calls, ensuring seamless orchestration and automation.

5. State and Session Management

State management is handled using the **Terraform state file**, which keeps track of the current infrastructure and ensures consistency between the defined configuration and actual resources. It helps in managing updates, deletions, and changes efficiently.

Session management is maintained through secure authentication mechanisms in the CI/CD pipeline and management interfaces, ensuring that only authorized users can access and perform operations on the system.

6. Caching

Caching is used to improve system performance and reduce deployment time. **Docker layer caching** is utilized to speed up image building by reusing unchanged layers. Additionally, **Terraform plugins and dependencies** are cached locally to avoid repeated downloads during provisioning. This results in faster execution of CI/CD pipelines and efficient resource utilization.

7. Functional Requirements

The functional requirements define the specific behaviors and services the Management Portal must provide to facilitate the IaC provisioning of the Telecom system.

- **User Authentication & Authorization:**
  The system should provide secure login functionality for users and enforce role-based access control. Only authorized users should be able to trigger deployments or modify infrastructure configurations.
- **Infrastructure Configuration Management:**
  The system should allow users to create, update, and manage infrastructure using Infrastructure as Code (IaC) scripts. It should support environment-specific configurations for flexible deployments.
- **Provisioning & Deployment:**
  The system should automatically provision infrastructure and deploy telecom services using Terraform and CI/CD pipelines. It should ensure consistent and repeatable deployments across environments.
- **Monitoring & Logging:**
  The system should provide real-time monitoring of infrastructure and services. It should maintain detailed logs of deployments and user activities for debugging and auditing purposes.
- **Service Management:**
  The system should enable users to start, stop, restart, and manage telecom microservices efficiently through a centralized interface.
- **Containerization & Orchestration:**
  The system should use Docker to containerize applications and Kubernetes to manage, scale, and ensure high availability of telecom services.

8. Non-Functional Requirements

- **Security:**
  The system should ensure secure handling of sensitive data such as credentials and API keys using encryption and access control mechanisms. Authentication and authorization must be enforced for all users.
- **Performance:**
  The system should provide fast and efficient infrastructure provisioning and deployment, minimizing delays in CI/CD pipelines and optimizing resource usage.
- **Scalability:**
  The system should support scaling of telecom services based on demand using Kubernetes, ensuring high availability and load balancing.
- **Reliability:**
  The system should ensure consistent and error-free deployments by minimizing manual intervention and maintaining infrastructure state through Terraform.
- **Maintainability:**
  The system should be modular and easy to update, allowing developers to modify components without affecting the entire system.
- **Availability:**
  The system should ensure continuous operation of telecom services with minimal downtime, even during updates or failures.

9. References

- Problem Statement: IaC Provisioning for Telecom System
- Docker Documentation: https://docs.docker.com/
- Kubernetes Documentation: https://kubernetes.io/docs/
- Jenkins Documentation: https://www.jenkins.io/doc/
- GitHub Repository: https://github.com/mohddilshadkhan61816-lab/-IaC-Provisioning-for-Telecomm-System